

Aavihotr Anand Jha

90119991

Assignment 4 – CONSTRAINTS, DATABASE DESIGN & NORMALIZATION

Q1.

Key Type	Example
Super Key	(loan_id, principal_amount)
Candidate Key	loan_id
Primary Key	loan_id
Foreign Key	customer_id, branch_id

(loan_id, principal_amount) is a super key because it uniquely identifies a loan, but it is not minimal. loan_id alone is already unique, so principal_amount is unnecessary.

Q2.

There are three reasons:

1. Email is not stable — customers can change their email address, requiring all referencing records to be updated.
2. Email can be NULL — not every customer necessarily has an email address, whereas a primary key cannot be NULL.
3. Email is large and repeated — an email string takes more space than an integer customer ID and makes indexes and joins larger.

Q3.

RESTRICT vs CASCADE

- RESTRICT → prevents deleting the parent row when child rows exist.
- CASCADE → deletes the related child rows automatically when the parent is deleted.

Relationship	Rule	Reason
account_txn → account	RESTRICT	Transaction history is financial/regulatory evidence.
card_reward_enrolment → credit_card	CASCADE	An enrolment has no meaning without its card.
loan → customer	RESTRICT	A customer with an outstanding loan should not simply disappear.
employee.manager_id → employee	SET NULL	If a manager leaves, the employees should remain.
loan_payment → loan	CASCADE	Loan payments have no meaning without the loan.

Q4.

Insert anomaly

An insert anomaly occurs when one fact cannot be inserted unless an unrelated fact is also available.

Example:

The Jaipur branch cannot be registered until it has its first customer because branch_customer_flat uses cust_id as its key.

Update anomaly

An update anomaly occurs when the same fact is stored multiple times and all copies must be updated correctly.

Example:

When Koramangala was renamed, only 1 of 4 rows was updated while the other three retained old spellings.

Delete anomaly

A delete anomaly occurs when deleting one fact unintentionally removes another fact that was stored in the same row.

Example:

Deleting customer 1027 removed their phone number, city and product list together with the account information.

Q5.

1. BRANCH ||-----o{ EMPLOYEE

Meaning:

- An employee belongs to exactly one branch.
- Therefore, employee.branch_id is mandatory.
- A branch can have zero, one or many employees.

Implementation:

employee.branch_id NOT NULL

FOREIGN KEY → branch

2. LOAN ||-----|{ LOAN_PAYMENT

Meaning:

- Every loan payment belongs to exactly one loan.
- Every loan must have at least one payment.
- A loan can have many payments.

loan_payment.loan_id should therefore be NOT NULL.

The requirement that every loan must have at least one payment cannot be enforced using an ordinary row-level CHECK; it must be guaranteed by the process that creates the loan and its payment schedule.

Q6.

A foreign key stores one value, so it can point to one parent row.

A many-to-many relationship requires one record to be associated with many records on the other side. A single foreign-key column cannot store multiple referenced values.

Therefore, a junction table is required.

For example:

customer_product

customer_id

product_id

The primary key should normally be:

PRIMARY KEY (customer_id, product_id)

This prevents the same customer from being enrolled in the same product more than once.

If the relationship itself has history, such as repeated enrolment, an additional attribute such as enrolled_on or a suitable surrogate key may be required.

Q7.

Normal Form	Rule	Anomaly Removed
1NF	Values must be atomic; no repeating groups; each row uniquely identifiable.	Prevents multiple values being stored in one column.
2NF	Must be in 1NF and no non-key column can depend on only part of a composite key.	Removes partial dependency and repeated parent facts.
3NF	Must be in 2NF and no non-key column depends on another non-key column.	Removes transitive/update anomalies.
BCNF	Every determinant must be a candidate key.	Removes remaining redundancy caused by overlapping candidate keys.

Q8.

A table with a single-column primary key cannot violate 2NF because 2NF deals with partial dependency.

A partial dependency occurs when a non-key attribute depends on only part of a composite key.

A single-column key has no "part", so a partial dependency is impossible.

Implication

If a schema uses surrogate single-column keys throughout, 2NF problems largely disappear. The main normalization concerns then become 1NF and 3NF.

Composite natural keys are where 2NF violations most commonly occur, especially in junction tables.

Q9.

Table	Violation	Fix
customer(customer_id, name, phone_1, phone_2, phone_3)	1NF – repeating group	Create customer_phone(customer_id, phone_type, phone_number)
order_line(order_id, product_id, quantity, product_name, product_price)	2NF – product details depend only on product_id	Move product details to product
employee(employee_id, name, branch_id, branch_name, branch_city)	3NF – branch details depend on branch_id	Move branch details to branch
account(account_id, customer_id, customer_name, balance)	3NF – customer name depends on customer_id	Move customer name to customer

Important distinction

order_line is a 2NF violation, not a 3NF violation, because product_id is part of the composite key.

Q10.

account.balance is derivable from account_txn, so storing it separately breaks the normalization principle because it is a derived value.

Balance enquiries happen extremely frequently and must be very fast. Recalculating the balance from the complete transaction history every time would be expensive.

A reconciliation control must periodically compare:

stored account.balance VS balance calculated from transactions

and raise an alert when they differ beyond an acceptable tolerance.

A derived value without reconciliation cannot be trusted.

Q11.

1.

```
SELECT branch_code,  
       COUNT(DISTINCT branch_ifsc) AS distinct_ifsc  
FROM branch_customer_flat  
GROUP BY branch_code  
HAVING COUNT(DISTINCT branch_ifsc) > 1;
```

Output

0 rows

This means no branch code has more than one IFSC in the current data.

2.

```
SELECT branch_code,  
       COUNT(DISTINCT branch_name) AS distinct_names  
FROM branch_customer_flat  
GROUP BY branch_code  
HAVING COUNT(DISTINCT branch_name) > 1;
```

Output

branch_code | distinct_names

BR03 | 4

Q12.

```
SELECT branch_code,  
       COUNT(*) AS customer_rows,  
       COUNT(*) - 1 AS redundant_copies,  
       SUM(LENGTH(branch_name)) AS bytes_stored,  
       MAX(LENGTH(branch_name)) AS bytes_if_stored_once  
FROM branch_customer_flat  
GROUP BY branch_code  
ORDER BY branch_code;
```

Repeated rows

Branch	Customer Rows	Redundant Copies
BR01	4	3
BR02	4	3
BR03	4	3
BR04	4	3
BR05	4	3
BR06	4	3

Branch	Customer Rows	Redundant Copies
BR07	3	2
BR08	3	2

Bank-wide storage

Stored currently = 382 bytes

Stored if once/branch = 134 bytes

So there are 248 redundant bytes in this small dataset.

But the important issue is not the storage saving. The real problem is that repeated values allowed the four inconsistent spellings of BR03 to exist.

Q13.

```
SELECT COUNT(*) AS rows,
       COUNT(DISTINCT cust_phone) AS distinct_phones
FROM branch_customer_flat;
```

Output

rows | distinct_phones

30 | 30

Since all 30 phone numbers are distinct, cust_phone is a candidate key in the current dataset.

A phone number is not stable: customers can change numbers, and numbers may later be reassigned by telecom operators. Therefore, phone uniqueness in this dataset is not a reliable identity rule.

Q14.

1. Create customer-account table

```
CREATE TABLE ex4_customer_account AS
SELECT cust_id,
       acct_no_1 AS acct_no,
       acct_type_1 AS acct_type,
       balance_1 AS balance
FROM branch_customer_flat
WHERE acct_no_1 IS NOT NULL
```

UNION ALL

```
SELECT cust_id,
       acct_no_2,
       acct_type_2,
       balance_2
FROM branch_customer_flat
WHERE acct_no_2 IS NOT NULL;
```

2. Create customer-product table

```
CREATE TABLE ex4_customer_product AS
SELECT f.cust_id,
       TRIM(s.product_code) AS product_code
FROM branch_customer_flat f
CROSS JOIN LATERAL
```

```
UNNEST(STRING_TO_ARRAY(f.products, ','))
AS s(product_code)
WHERE f.products <> 'NONE';
```

3. Check row counts

```
SELECT
  (SELECT COUNT(*) FROM ex4_customer_account) AS accounts,
  (SELECT COUNT(*) FROM ex4_customer_product) AS product_holdings;
```

The solution document reports:

accounts = 36

and discusses 59 product holdings: 27 cards + 20 fixed deposits + 12 loans.

Q15.

1. Create branch table

```
CREATE TABLE ex4_branch AS
```

```
SELECT DISTINCT
```

```
  branch_code,
```

```
  branch_ifsc,
```

```
  branch_city
```

```
FROM branch_customer_flat;
```

This produces 8 branches.

2. Add branch name

```
ALTER TABLE ex4_branch
```

```
ADD COLUMN branch_name VARCHAR(60);
```

3. Choose the surviving branch name

```
UPDATE ex4_branch b
```

```
SET branch_name =
```

```
(
```

```
  SELECT f.branch_name
```

```
  FROM branch_customer_flat f
```

```
  WHERE f.branch_code = b.branch_code
```

```
  GROUP BY f.branch_name
```

```
  ORDER BY COUNT(*) DESC,
```

```
          f.branch_name
```

```
  LIMIT 1
```

```
);
```

4. Make it mandatory

```
ALTER TABLE ex4_branch
```

```
ALTER COLUMN branch_name SET NOT NULL;
```

5. Add primary key

```
ALTER TABLE ex4_branch
```

```
ADD CONSTRAINT pk_ex4_branch
```

```
PRIMARY KEY (branch_code);
```

Create customer table

```
CREATE TABLE ex4_customer AS
```

```
SELECT cust_id,
```

```
       cust_name,
```

```
       cust_phone,
```

```
       cust_city,
```

```
       branch_code
```

```
FROM branch_customer_flat;
```

```
ALTER TABLE ex4_customer
```

```
ADD CONSTRAINT pk_ex4_customer
```

```
PRIMARY KEY (cust_id);
```

```
ALTER TABLE ex4_customer
```

```
ADD CONSTRAINT fk_ex4_customer_branch
```

```
FOREIGN KEY (branch_code)
```

```
REFERENCES ex4_branch(branch_code);
```

Rule for BR03

The rule used is:

Choose the most frequent spelling; if tied, choose alphabetically.

All four BR03 spellings occur once, so the alphabetical tie-break selects Koramangala.

However, the solution explicitly says that this is only a heuristic. In a real banking cleanup, the authoritative branch name should come from the branch licence register or RBI IFSC directory.

Q16.

1. Insert anomaly

A branch can now exist without a customer.

```
BEGIN;
```

```
INSERT INTO ex4_branch
```

```
(
```

```
    branch_code,
```

```
    branch_ifsc,
```

```
    branch_city,
```

```
    branch_name
```

```
)
```

```
VALUES
```

```
(
```

```
    'BR09',
```

```
    'ICIC0000709',
```

```
    'Jaipur',
```

```
    'Malviya Nagar'
```

```
);
```

```
SELECT COUNT(*)
```

```
FROM ex4_branch;
```

```
ROLLBACK;
```

Output

count

9

The branch can now exist independently.

2. Update anomaly

Only one branch row needs to be updated.

BEGIN;

UPDATE ex4_branch

SET branch_name = 'Koramangala Main'

WHERE branch_code = 'BR03';

Output

UPDATE 1

Then:

SELECT branch_code, branch_name

FROM ex4_branch

WHERE branch_code = 'BR03';

ROLLBACK;

Only one row is affected.

3. Delete anomaly

Deleting an account does not delete the customer.

BEGIN;

DELETE FROM ex4_customer_account

WHERE acct_no = 'ICIC0000009032';

SELECT cust_id,

cust_name,

cust_phone

FROM ex4_customer

WHERE cust_id = 1027;

ROLLBACK;

The customer record still exists, including the phone number.

Therefore, deleting the account no longer deletes unrelated customer information.

Q17.

Locker table

CREATE TABLE ex4_locker

(

locker_id INTEGER NOT NULL,

branch_id INTEGER NOT NULL,

locker_number VARCHAR(10) NOT NULL,

locker_size VARCHAR(10) NOT NULL,

```
annual_rent NUMERIC(10,2) NOT NULL,  
status VARCHAR(12) DEFAULT 'AVAILABLE' NOT NULL,
```

```
CONSTRAINT pk_ex4_locker  
PRIMARY KEY (locker_id),
```

```
CONSTRAINT uq_ex4_locker  
UNIQUE (branch_id, locker_number),
```

```
CONSTRAINT fk_ex4_locker_branch  
FOREIGN KEY (branch_id)  
REFERENCES branch(branch_id)  
ON DELETE RESTRICT,
```

```
CONSTRAINT ck_ex4_locker_size  
CHECK (locker_size IN ('SMALL','MEDIUM','LARGE')),
```

```
CONSTRAINT ck_ex4_locker_rent  
CHECK (annual_rent > 0),
```

```
CONSTRAINT ck_ex4_locker_status  
CHECK (status IN  
('AVAILABLE','ALLOTTED','SURRENDERED'))
```

```
);
```

Locker agreement table

```
CREATE TABLE ex4_locker_agreement
```

```
(  
  agreement_id INTEGER NOT NULL,  
  locker_id INTEGER NOT NULL,  
  customer_id INTEGER NOT NULL,  
  start_date DATE NOT NULL,  
  end_date DATE NULL,  
  status VARCHAR(12) DEFAULT 'ACTIVE' NOT NULL,
```

```
CONSTRAINT pk_ex4_locker_agreement  
PRIMARY KEY (agreement_id),
```

```
CONSTRAINT fk_ex4_la_locker  
FOREIGN KEY (locker_id)  
REFERENCES ex4_locker(locker_id)  
ON DELETE RESTRICT,
```

```
CONSTRAINT fk_ex4_la_customer  
FOREIGN KEY (customer_id)  
REFERENCES customer(customer_id)  
ON DELETE RESTRICT,
```

```

CONSTRAINT ck_ex4_la_dates
    CHECK (end_date IS NULL OR end_date > start_date),

CONSTRAINT ck_ex4_la_status
    CHECK (status IN
        ('ACTIVE','CLOSED','SURRENDERED'))
);

```

Locker visit table

```

CREATE TABLE ex4_locker_visit
(
    visit_id BIGINT NOT NULL,
    agreement_id INTEGER NOT NULL,
    visited_at TIMESTAMP NOT NULL,
    supervised_by INTEGER NOT NULL,

    CONSTRAINT pk_ex4_locker_visit
        PRIMARY KEY (visit_id),

    CONSTRAINT fk_ex4_lv_agreement
        FOREIGN KEY (agreement_id)
        REFERENCES ex4_locker_agreement(agreement_id)
        ON DELETE RESTRICT,

    CONSTRAINT fk_ex4_lv_employee
        FOREIGN KEY (supervised_by)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
);

```

Q18.

Question	Answer
Which entity is weak?	ex4_locker_visit, because a visit has no meaning without the agreement under which it occurred.
Which relationship is many-to-many?	Customer ↔ Locker over time.
How is it resolved?	ex4_locker_agreement acts as the relationship/junction entity.
Which FK is nullable?	None. end_date is nullable because an active agreement may not have an end date yet.
CHECK rule 1	end_date IS NULL OR end_date > start_date
CHECK rule 2	annual_rent > 0

Important additional rule

The rule:

One locker can have only one active customer at a time

cannot be implemented using a normal CHECK, because it involves multiple rows.

In PostgreSQL, a partial unique index can enforce it:

```
CREATE UNIQUE INDEX uq_ex4_la_active
ON ex4_locker_agreement (locker_id)
WHERE status = 'ACTIVE';
```

Q19.

The date constraint is already defined as:

```
CONSTRAINT ck_ex4_la_dates
CHECK (end_date IS NULL OR end_date > start_date)
```

Test invalid dates

```
INSERT INTO ex4_locker_agreement
```

```
(
  agreement_id,
  locker_id,
  customer_id,
  start_date,
  end_date,
  status
```

```
)
```

```
VALUES
```

```
(
  9001,
  1,
  1001,
  DATE '2026-04-01',
  DATE '2026-03-01',
  'ACTIVE'
```

```
);
```

Expected error

```
ERROR: new row for relation "ex4_locker_agreement"
violates check constraint
```

```
"ck_ex4_la_dates"
```

Valid NULL end date

```
INSERT INTO ex4_locker_agreement
```

```
(
  agreement_id,
  locker_id,
  customer_id,
  start_date,
  end_date,
  status
```

```
)
```

```
VALUES
```

```
(
  9002,
  1,
```

```
1001,  
DATE '2026-04-01',  
NULL,  
'ACTIVE'  
);
```

Output

```
INSERT 0 1
```

NULL is allowed because an active agreement can legitimately have no end date yet.

Q20.

Create summary table

```
CREATE TABLE ex4_branch_daily_summary  
(  
    branch_id INTEGER NOT NULL,  
    summary_date DATE NOT NULL,  
    txn_count INTEGER NOT NULL,  
    total_credits NUMERIC(15,2) NOT NULL,  
    total_debits NUMERIC(15,2) NOT NULL,  
  
    CONSTRAINT pk_ex4_bds  
        PRIMARY KEY (branch_id, summary_date),  
  
    CONSTRAINT fk_ex4_bds_branch  
        FOREIGN KEY (branch_id)  
        REFERENCES branch(branch_id)  
);
```

Populate January 2026 data

```
INSERT INTO ex4_branch_daily_summary  
(  
    branch_id,  
    summary_date,  
    txn_count,  
    total_credits,  
    total_debits  
)  
SELECT  
    a.branch_id,  
    CAST(t.txn_date AS DATE),  
    COUNT(*),  
  
    SUM(  
        CASE  
            WHEN t.txn_type = 'CREDIT'  
            THEN t.amount  
            ELSE 0  
        END  
    ),  
,
```

```

SUM(
  CASE
    WHEN t.txn_type = 'DEBIT'
    THEN t.amount
    ELSE 0
  END
)

FROM account_txn t
JOIN account a
  ON a.account_id = t.account_id

WHERE t.status = 'SUCCESS'
AND t.txn_date >= DATE '2026-01-01'
AND t.txn_date < DATE '2026-02-01'

GROUP BY
  a.branch_id,
  CAST(t.txn_date AS DATE);

```

Primary key

(branch_id, summary_date)

This is appropriate because the grain of the table is one branch per day.

The exercise asks for a historical closing deposit total, but the available account.balance is only the current balance and has no historical snapshots.

Therefore, the January 2026 closing balance cannot be correctly reconstructed from the available tables.

It should be captured by a nightly balance snapshot going forward rather than invented from current balances.

Q21.

Use a FULL OUTER JOIN so that missing dates on either side are also detected.

```

SELECT
  s.branch_id,
  s.summary_date,

  s.txn_count AS summary_count,
  t.txn_count AS live_count,

  s.total_credits AS summary_credits,
  t.total_credits AS live_credits

FROM ex4_branch_daily_summary s

FULL OUTER JOIN
(
  SELECT

```

```

a.branch_id,
CAST(t.txn_date AS DATE) AS summary_date,
COUNT(*) AS txn_count,

SUM(
  CASE
    WHEN t.txn_type = 'CREDIT'
    THEN t.amount
    ELSE 0
  END
) AS total_credits

FROM account_txn t
JOIN account a
  ON a.account_id = t.account_id

WHERE t.status = 'SUCCESS'
AND t.txn_date >= DATE '2026-01-01'
AND t.txn_date < DATE '2026-02-01'

GROUP BY
  a.branch_id,
  CAST(t.txn_date AS DATE)

) t
ON t.branch_id = s.branch_id
AND t.summary_date = s.summary_date

WHERE
  s.txn_count IS DISTINCT FROM t.txn_count
OR ABS(
  COALESCE(s.total_credits, 0)
  -
  COALESCE(t.total_credits, 0)
) > 0.01;

```

Expected output

0 rows

This means the summary agrees with the transactional data.

If rows are returned

A non-empty result could indicate:

1. A summary day exists but not in the source.
2. A source day was missed by the summary.
3. More transactions arrived after the summary was generated.
4. The transaction totals differ due to rounding/type differences.

FULL OUTER JOIN is important because an inner join would miss an entire day that exists on only one side. IS DISTINCT FROM is used because it handles NULL comparisons safely.

Q22.

We should not use ON DELETE CASCADE for financial records such as account_txn → account or account → customer.

With both cascades enabled, deleting one customer could automatically delete all their accounts and every transaction associated with those accounts.

The dangerous part is that the DELETE would succeed without an error, leaving no immediate indication that financial history had been removed.

For example, a support engineer could accidentally execute a customer cleanup script with an overly broad WHERE condition. With RESTRICT, the operation would fail and require investigation. With CASCADE, the data could disappear silently.

Required rule

Use:

ON DELETE RESTRICT

or:

ON DELETE NO ACTION

for financial/audit records such as:

- accounts
- transactions
- loans
- loan payments
- deposits
- statements

CASCADE can be considered for genuinely dependent non-financial data such as preferences, notification settings or reward enrolments.

Q23.

Argument 1 — The limit of three is arbitrary

Using:

phone_1

phone_2

phone_3

assumes that customers will never need a fourth number.

If that changes, the production table and every query, screen and report using phone numbers would need modification.

A separate customer_phone table can support any number of phone numbers without changing the schema.

Argument 2 — It is difficult to query correctly

For example:

Find customers with a verified mobile number.

With three columns, the query would have to check:

phone_1 OR phone_2 OR phone_3

There is also nowhere naturally to store attributes such as:

- phone type
- verified status
- preferred number

These attributes belong to the individual phone record.

When would three columns be reasonable?

If the business rule is genuinely:

one mobile number

+

optionally one landline

with fixed meanings and no additional attributes, then:

mobile_number

landline_number

are two distinct customer attributes rather than interchangeable repeating columns.

The key test is whether the columns are interchangeable.

Q24.

Why is it acceptable in the warehouse?

A warehouse fact table may contain 400 million rows, and the data is loaded through a controlled ETL process.

If branch_name is stored directly in the fact table, every report does not need to repeatedly join 400 million rows to the branch dimension.

This can provide a significant performance benefit.

Why would this not be acceptable in the customer table?

The customer table is continuously updated by multiple applications and branch staff.

If branch information is duplicated there, one row could be updated while another is not, producing inconsistent values — exactly like the four BR03 spellings.

What makes warehouse denormalization safe?

The important control is:

The ETL process is the single writer.

It derives branch_name from the authoritative branch dimension during loading.

Nobody independently edits the duplicated value.

The warehouse row also represents a point-in-time record, so retaining the historical branch name can be intentional.

Q25.

I would not accept "pick the most frequent spelling" as a blanket rule.

The most frequent spelling has no guarantee of being the correct one. If three rows contained an incorrect spelling and only one contained the correct spelling, automatically choosing the most frequent value would permanently encode the error.

Better approach

1. Run a discrepancy query across all eight branches.
2. Automatically fix differences that are only formatting differences, such as whitespace or case.
3. For substantive differences, compare against an authoritative source such as the RBI IFSC directory.
4. Send only unresolved cases to a human reviewer.
5. Record the original value, selected value and reason in a mapping table.

If the deadline is extremely tight, it is safer to leave the names unchanged temporarily than to make an unaudited decision that destroys the original evidence.

Q26.

The argument is partly correct: modern hardware makes indexed joins relatively cheap, storage is inexpensive, and there are situations where denormalization is useful.

However, normalization is not primarily a storage optimization.

The real cost it prevents is incorrect and inconsistent data.

If the same fact is stored in four places, every application that updates that fact must correctly update all four copies. Eventually one update can miss one copy, creating two contradictory answers to the same question.

That is what happened with the four spellings of Koramangala.

The 248 bytes saved in this small dataset are practically irrelevant. The important benefit is that storing the branch fact once makes conflicting values impossible rather than merely unlikely.

When is denormalization reasonable?

When:

- one controlled process writes the data,
- the data is not independently updated afterward,
- and the duplicated value is intentionally maintained.

That is why denormalization can be appropriate in a controlled reporting warehouse.

Q27.

Ticket ICICI-4577

Dear Mr Iyer,

Ticket ICICI-4577 is closed. The customer master previously stored customer details, branch information, account details and product information together in one wide row.

This meant the same branch information was copied into many customer records instead of being maintained once for the branch.

As a result, Koramangala appeared with four different spellings because different records could contain different values.

The same design also meant that deleting an account could remove the customer's phone number and other customer information stored in the same row.

The replacement stores each fact in the record for the thing it describes.

Branch information is maintained in the branch table, customer information in the customer table, and accounts in their own table linked to the customer.

Because the branch name now exists in one controlled location, conflicting branch spellings cannot be created through customer records.

Because an account is stored separately, closing an account removes only the account and does not remove the customer's personal information.

Branches will therefore maintain branch details centrally rather than changing them from customer screens.

Customers can also have any number of accounts and products instead of being restricted to fixed account fields and a text list.

Finally, the historical spelling issue was resolved using the authoritative branch record, with the mapping retained for audit purposes.